

Banking Case Study 1

Validating Bank Account Balances After Debit and Credit Transactions

This case study demonstrates how financial accuracy can be validated in a digital banking system by testing how customer account balances are affected after debit transactions, credit transactions, transaction fees and transaction history updates.

The purpose of this case study is to show how regulated transactional testing experience can transfer into banking and FinTech QA. The same testing discipline used for balances, payouts and transactional workflows can be applied to banking systems where account balances, transaction records, statements, audit trails and backend data must remain accurate and traceable.

This case study focuses on a fictional online banking platform. No real bank data, client data or confidential system information is used.

1. Business Problem

In digital banking, every debit and credit transaction must update the customer's account balance correctly.

A customer may perform actions such as transferring money, receiving money, paying an account, withdrawing money, reversing a failed transaction or being charged a transaction fee.

Each action changes the account balance. If the balance is calculated incorrectly, the impact can be serious.

- customer complaints
- incorrect statements
- failed reconciliation
- audit issues
- regulatory risk
- financial loss
- loss of customer trust

The system must therefore ensure that the customer facing balance, transaction history, statement record, admin view, API response and database record all reflect the same financial truth.

2. Why This Case Study Fits My Background

This case study is based on experience testing regulated transactional systems.

iGaming Testing Area	Banking Equivalent
Player opening balance	Customer opening account balance
Wager deduction	Debit transaction
Payout credit	Credit transaction
Game history	Bank transaction history
Platform history	Mobile banking or web banking history
Back office history	Internal banking admin view
Game transaction record	Core banking database record
Certification evidence	Audit and compliance evidence

The logic is different, but the testing mindset is very similar. Both environments require accurate balances, transaction traceability, clear audit trails, backend validation, strong defect reporting, regulatory awareness and evidence based testing.

3. Business Risk

Risk Area	Possible Impact
Incorrect account balance	Customer may see the wrong available balance
Wrong debit amount	Customer may be overcharged or undercharged
Missing credit	Customer may not receive expected funds
Incorrect transaction fee	Bank charges may be inaccurate
Statement mismatch	Customer statement may not match backend records
Audit trail issue	Transaction investigation may be difficult
Duplicate transaction	Customer may be debited more than once
Failed reversal	Failed transaction may not be corrected
Compliance risk	Bank may fail internal or external audit checks
Customer trust risk	Customer may lose confidence in the banking platform

4. Test Objective

The objective of this case study is to confirm that the banking system correctly updates a customer's account balance after a debit and credit transaction.

Validation Area	Objective
Opening balance	Confirm the starting balance before the transaction
Debit amount	Confirm the correct amount is deducted
Credit amount	Confirm the correct amount is added
Transaction fee	Confirm fees are applied correctly where applicable
Closing balance	Confirm the final balance is mathematically correct
Transaction history	Confirm the customer facing history displays correctly
Statement record	Confirm the official statement reflects the transaction
Admin portal	Confirm internal records match customer facing records
Database	Confirm backend values are stored correctly
API response	Confirm API response matches expected transaction data
Audit trail	Confirm transaction ID, timestamp and status are traceable

5. Test Scope

In Scope

Area	Description
Customer login	Confirm that the customer can access the banking platform
Account balance display	Confirm that the opening balance displays correctly
Debit transaction	Confirm that money is deducted correctly
Credit transaction	Confirm that money is added correctly
Transaction fee	Confirm that the correct fee is applied
Closing balance	Confirm that the final balance is correct
Transaction history	Confirm that transaction details appear correctly
Statement validation	Confirm that statement records match the transaction
Admin portal validation	Confirm that internal records match customer records
API validation	Confirm that transaction API responses are accurate
SQL validation	Confirm that database records match expected values
Reconciliation	Compare all sources for consistency
Defect reporting	Log any mismatch with evidence and risk

Out of Scope

Area	Reason
Real banking systems	This is a fictional portfolio case study
Production customer data	No real customer data is used
Security penetration testing	This case study focuses on financial accuracy
Fraud detection rules	Fraud testing is not covered in this case study
Full regulatory legal review	The focus is QA validation, not legal interpretation
Loan or interest calculations	This case study focuses on basic account transactions

6. Test Environment

Item	Example Value
Application type	Demo digital banking platform
Platform	Web banking or mobile banking
Environment	QA environment
Customer account	Fictional funded account
Account type	Savings account
Currency	ZAR
Browser	Chrome, Edge or Firefox
API tool	Postman
Database	PostgreSQL or MySQL
Defect tool	JIRA style defect tracking
Evidence	Screenshots, API responses, SQL results and reconciliation tables

7. Test Data

Field	Value
Customer ID	CUST1001
Account number	ACC1001
Account type	Savings
Opening balance	R5 000.00
Debit amount	R750.00
Credit amount	R1 200.00
Transaction fee	R5.00
Expected balance after debit	R4 245.00
Expected balance after credit	R5 445.00
Transaction status	Completed
Currency	ZAR

8. Balance Calculation Logic

Debit Transaction Formula

For a debit transaction with a fee:

Closing Balance = Opening Balance minus Debit Amount minus Transaction Fee

Example: R5 000.00 minus R750.00 minus R5.00 = R4 245.00

Credit Transaction Formula

For a credit transaction after the debit has already happened:

New Closing Balance = Current Balance plus Credit Amount

Example: R4 245.00 plus R1 200.00 = R5 445.00

9. End to End Transaction Flow

Step	Banking Event	Expected Result
1	Customer logs into banking platform	Login is successful
2	Customer views account balance	Opening balance is R5 000.00
3	Customer makes a R750.00 payment	Debit transaction is created
4	System applies R5.00 transaction fee	Fee is deducted correctly
5	Balance is updated	Balance becomes R4 245.00
6	Transaction history is checked	Debit and fee are displayed
7	Admin portal is checked	Internal record matches customer view
8	Database is checked	Backend record stores correct values
9	Customer receives R1 200.00 credit	Credit transaction is created
10	Balance is updated again	Balance becomes R5 445.00
11	Statement is checked	Both transactions appear correctly
12	API response is checked	API data matches UI and database

10. Detailed Test Case 1

Test Case ID

TC BANK BAL 001

Test Case Title

Verify that a debit transaction updates the customer’s account balance correctly

Test Objective

To confirm that the system correctly deducts a debit transaction amount and transaction fee from the customer’s account balance.

Preconditions

Precondition	Description
Customer account exists	A valid test customer account is available
Account is funded	Customer has an opening balance of R5 000.00
Banking platform is available	Customer can access the QA banking environment
Payment feature is enabled	Customer can perform a debit transaction
Transaction fee rule exists	R5.00 fee applies to this transaction type
Database access is available	Tester can validate backend transaction records
API access is available	Tester can validate transaction response

Test Steps

Step	Action	Expected Result
1	Login using the test customer account	Customer logs in successfully
2	Open the account dashboard	Savings account is displayed
3	Confirm opening balance	Balance displays R5 000.00
4	Start a payment transaction	Payment screen opens successfully
5	Enter payment amount of R750.00	Amount is accepted
6	Confirm payment details	Debit amount and fee are displayed
7	Submit payment	Payment is processed successfully
8	Check updated account balance	Balance displays R4 245.00
9	Open transaction history	Debit transaction and fee are displayed
10	Check admin portal record	Internal record matches customer history
11	Query database transaction record	Stored values match expected values
12	Validate API response	API response matches UI and database
13	Capture evidence	Screenshots, SQL result and API response are saved
14	Mark result	Test passes if all values match

Expected Result

The system should deduct both the debit amount and the transaction fee.

Field	Expected Value
Opening balance	R5 000.00
Debit amount	R750.00
Transaction fee	R5.00
Expected closing balance	R4 245.00

11. Detailed Test Case 2

Test Case ID

TC BANK BAL 002

Test Case Title

Verify that a credit transaction updates the customer's account balance correctly

Test Objective

To confirm that the system correctly adds an incoming credit amount to the customer's account balance.

Preconditions

Precondition	Description
Customer account exists	Test account is available
Debit transaction already completed	Current balance is R4 245.00
Incoming credit is available	R1 200.00 credit transaction can be processed
Transaction logging is enabled	History and statement records are available
Database access is available	Tester can validate backend data

Test Steps

Step	Action	Expected Result
1	Login using the test customer account	Customer logs in successfully
2	Open account dashboard	Current balance displays R4 245.00
3	Process incoming credit of R1 200.00	Credit transaction is completed
4	Refresh account dashboard	Balance updates successfully
5	Confirm new account balance	Balance displays R5 445.00
6	Open transaction history	Credit transaction is displayed
7	Open statement view	Credit transaction appears correctly
8	Check admin portal record	Internal record matches customer view
9	Query database transaction record	Stored values match expected values
10	Validate API response	API response confirms completed credit
11	Capture evidence	Screenshots, SQL result and API response are saved
12	Mark result	Test passes if all values match

Expected Result

Field	Expected Value
Current balance before credit	R4 245.00
Credit amount	R1 200.00
Expected closing balance	R5 445.00

12. Reconciliation Table

Field	Expected	Mobile Banking	Statement	Admin Portal	API Response	Database	Result
Opening balance	R5 000.00	R5 000.00	R5 000.00	R5 000.00	R5 000.00	R5 000.00	Pass
Debit amount	R750.00	R750.00	R750.00	R750.00	R750.00	R750.00	Pass
Transaction fee	R5.00	R5.00	R5.00	R5.00	R5.00	R5.00	Pass
Balance after debit	R4 245.00	R4 245.00	R4 245.00	R4 245.00	R4 245.00	R4 245.00	Pass
Credit amount	R1 200.00	R1 200.00	R1 200.00	R1 200.00	R1 200.00	R1 200.00	Pass
Final balance	R5 445.00	R5 445.00	R5 445.00	R5 445.00	R5 445.00	R5 445.00	Pass

13. SQL Validation Examples

Query 1: Validate customer account balance

```
SELECT
    customer_id,
    account_number,
    account_type,
    available_balance,
    ledger_balance,
    currency,
    updated_at
FROM bank_accounts
WHERE account_number = 'ACC1001';
```

Expected SQL Result

customer_id	account_number	account_type	available_balance	ledger_balance	currency
CUST1001	ACC1001	Savings	5445.00	5445.00	ZAR

Query 2: Validate debit transaction

```
SELECT
    transaction_id,
    account_number,
    transaction_type,
    amount,
    fee_amount,
    transaction_status,
    balance_after_transaction,
    created_at
FROM bank_transactions
WHERE account_number = 'ACC1001'
AND transaction_type = 'DEBIT'
```

ORDER BY created_at DESC;

Expected SQL Result

transaction_id	transaction_type	amount	fee_amount	status	balance_after_transaction
TXN2001	DEBIT	750.00	5.00	COMPLETED	4245.00

Query 3: Validate credit transaction

```
SELECT
    transaction_id,
    account_number,
    transaction_type,
    amount,
    transaction_status,
    balance_after_transaction,
    created_at
FROM bank_transactions
WHERE account_number = 'ACC1001'
AND transaction_type = 'CREDIT'
ORDER BY created_at DESC;
```

Expected SQL Result

transaction_id	transaction_type	amount	status	balance_after_transaction
TXN2002	CREDIT	1200.00	COMPLETED	5445.00

14. API Validation Examples

API Test 1: Get Account Balance

Request

GET /api/accounts/ACC1001/balance

Expected Response

```
{
  "customerId": "CUST1001",
  "accountNumber": "ACC1001",
  "availableBalance": 5445.00,
  "ledgerBalance": 5445.00,
  "currency": "ZAR",
  "status": "ACTIVE"
}
```

API Assertions

Assertion	Expected Result
Status code	200 OK
accountNumber	ACC1001
availableBalance	5445.00
ledgerBalance	5445.00
currency	ZAR
status	ACTIVE

API Test 2: Get Transaction History

Request

GET /api/accounts/ACC1001/transactions

Expected Response Example

```
{
  "accountNumber": "ACC1001",
  "transactions": [
    {
      "transactionId": "TXN2002",
      "type": "CREDIT",
      "amount": 1200.00,
      "status": "COMPLETED",
      "balanceAfterTransaction": 5445.00
    },
    {
      "transactionId": "TXN2001",
      "type": "DEBIT",
      "amount": 750.00,
      "feeAmount": 5.00,
      "status": "COMPLETED",
      "balanceAfterTransaction": 4245.00
    }
  ]
}
```

API Assertions

Assertion	Expected Result
Status code	200 OK
First transaction type	CREDIT
Credit amount	1200.00
Credit status	COMPLETED
Second transaction type	DEBIT

Debit amount	750.00
Fee amount	5.00
Debit status	COMPLETED

15. Negative Test Scenarios

Test Case ID	Scenario	Expected Result
TC BANK BAL 003	Customer attempts debit greater than available balance	Transaction is rejected
TC BANK BAL 004	Duplicate debit request is submitted	Duplicate transaction is prevented
TC BANK BAL 005	Transaction fee is missing	Defect is logged
TC BANK BAL 006	Balance updates in UI but not database	Defect is logged
TC BANK BAL 007	Database updates but statement does not update	Defect is logged
TC BANK BAL 008	API returns incorrect balance	Defect is logged
TC BANK BAL 009	Transaction status remains pending after completion	Defect is logged
TC BANK BAL 010	Credit is applied twice	Duplicate credit defect is logged
TC BANK BAL 011	Failed debit still reduces balance	Critical defect is logged
TC BANK BAL 012	Reversal does not restore original balance	Critical defect is logged

16. Sample Defect Report

Defect ID	DEF BANK BAL 001
Defect Title	Customer facing balance does not match backend balance after debit transaction
Severity	High
Priority	High
Environment	QA Banking Web Build 1.0.3
Module	Account Balance and Transaction History
Related Test Case	TC BANK BAL 001

Description

After completing a debit transaction of R750.00 with a R5.00 transaction fee, the customer facing balance does not match the backend balance stored in the database.

The expected closing balance is R4 245.00. The database stores the correct value, but the mobile banking dashboard displays R4 250.00, which means the transaction fee is not reflected in the customer facing balance.

Steps to Reproduce

Step	Action
1	Login using customer account CUST1001
2	Confirm opening balance is R5 000.00
3	Start a payment transaction
4	Enter debit amount of R750.00
5	Confirm the R5.00 transaction fee
6	Submit the payment
7	Check the customer facing account balance
8	Query the database record for the transaction
9	Compare dashboard balance with database balance

Expected Result

The customer facing balance should display R4 245.00.

Calculation: R5 000.00 minus R750.00 minus R5.00 = R4 245.00

Actual Result

The customer facing dashboard displays R4 250.00. The database displays R4 245.00.

Evidence

Evidence	Description
Screenshot 1	Opening balance of R5 000.00
Screenshot 2	Debit amount of R750.00
Screenshot 3	Transaction fee of R5.00
Screenshot 4	Dashboard balance showing R4 250.00
SQL result	Database balance showing R4 245.00
API response	API balance value used for comparison
Reconciliation table	UI and database mismatch highlighted

Business Impact

The customer may see an incorrect available balance after completing a payment. This may cause confusion, overspending, customer complaints and loss of trust in the banking platform.

Compliance Risk

The system does not display consistent financial information across customer facing and backend records. This creates an audit and reconciliation risk because the customer view does not match the official backend record.

Recommendation

The balance display logic should be reviewed to confirm whether the customer dashboard is excluding transaction fees when calculating the available balance. The fix should ensure that the dashboard, transaction history, statement, API response and database all reflect the same closing balance.

The defect should remain open until the transaction is retested successfully across all record sources.

17. Test Execution Summary

Metric	Count
Total test cases	12
Passed	9
Failed	1
Blocked	0
Not run	2
Defects logged	1
High severity defects	1
Critical defects	0

18. Summary of Findings

Testing confirmed that the banking system correctly processed the debit amount, credit amount and backend transaction records. However, one high severity issue was identified where the customer facing dashboard balance did not include the transaction fee after a debit transaction.

The backend database calculated the correct balance, but the customer facing dashboard displayed a different balance. This creates a mismatch between the customer view and backend financial record.

19. Final QA Recommendation

Based on the executed test scope, the product should not proceed to release until the balance display discrepancy has been resolved and retested.

The corrected build should be validated by repeating the debit transaction and confirming that the same closing balance appears across the customer dashboard, transaction history, statement view, admin portal, API response and database record.

Once all sources show the same balance, the defect can be closed.

20. Skills Demonstrated

Skill	How It Was Demonstrated
Functional testing	Tested login, account dashboard, payment flow and transaction history
Financial validation	Verified debit, credit, fee and balance calculations
SQL validation	Queried backend account and transaction records
API testing	Validated account balance and transaction history responses
Reconciliation	Compared UI, statement, admin, API and database records
Defect reporting	Logged a clear defect with steps, evidence and impact
Risk analysis	Explained customer, business and compliance impact
Test documentation	Created test cases, reconciliation tables and summary report
Banking domain thinking	Applied financial accuracy testing to digital banking

21. Portfolio Disclaimer

All examples in this case study are fictional and created for portfolio purposes. No real banking system, customer account, employer system, client system or confidential data is used. The case study is designed to demonstrate transferable QA skills from regulated transactional testing into banking and FinTech quality assurance.